

Rekomendasi Tool Baru untuk AI Relational Data Seeder

Untuk @berthojoris/mcp-mysql-server agar agent seperti Cursor atau Droid CLI bisa membuat dummy data pada tabel berelasi FK dengan lebih cepat, aman, dan akurat.

Ringkasan keputusan

Verdict: MCP saat ini sudah kuat sebagai access layer MySQL untuk agent AI, tetapi untuk case “buat dummy data otomatis pada tabel yang punya foreign key” sebaiknya ditambah category khusus seed_operations. Empat tool inti yang paling disarankan adalah plan_seed_data, generate_seed_preview, execute_seed_plan, dan validate_seed_integrity.

Alasannya sederhana: schema discovery, FK discovery, bulk insert, custom SQL, dan transaction memang sudah tersedia. Namun logika seeding relasional - dependency order, mapping parent-child, generator value, validasi constraint, rollback, dan post-check - masih terlalu banyak diserahkan ke LLM. Tool khusus akan membuat perilaku lebih repeatable, lebih aman, dan lebih mudah diuji.

Dasar penilaian

- Paket ini dideskripsikan sebagai MCP server MySQL production-ready dengan dynamic per-project permissions; versi yang terindeks saat penyusunan dokumen ini adalah 1.40.7.
- Dokumentasi menyebut category/tool yang sudah relevan: database_discovery, crud_operations, bulk_operations, custom_queries, transaction_management, constraint_management, dan analysis.
- Tool yang sudah mendukung fondasi seeding antara lain get_all_tables_relationships, get_database_summary, get_schema_rag_context, bulk_insert, execute_write_query, begin_transaction, commit_transaction, rollback_transaction, dan execute_in_transaction.

Sumber utama: GitHub repository berthojoris/mysql-mcp dan index paket npm/npmx yang menampilkan metadata package, versi, setup, permission, serta kategori tool. Tautan lengkap ada di bagian Referensi.

1. Gap utama saat ini

Agent sebenarnya sudah bisa melakukan seeding dengan tool yang ada, tetapi keberhasilannya sangat bergantung pada prompt dan kualitas reasoning agent. Untuk tabel sederhana, ini cukup. Untuk schema produksi yang punya banyak FK, unique constraint, enum/status, tenant_id, trigger, atau aturan domain, hasilnya bisa tidak konsisten.

Area	Kondisi sekarang	Risiko	Perbaikan yang disarankan
Dependency FK	Bisa dibaca via relationship/schema tools.	Agent bisa salah menentukan urutan parent -> child.	Tambahkan plan_seed_data untuk membuat dependency graph dan execution order.
Data dummy	LLM membuat value berdasarkan nama kolom dan tipe data.	Value status/type/role bisa salah secara domain.	Tambahkan infer_seed_rules atau seed_rules dalam plan.
Preview	Bergantung pada agent apakah mau menampilkan preview.	User bisa tidak sadar data yang akan diinsert.	Tambahkan generate_seed_preview dengan require_confirmation.
Execution	Bisa via bulk_insert/execute_write_query + transaction.	Jika prompt kurang tegas, agent bisa insert tanpa transaction.	Tambahkan execute_seed_plan yang default transaction + rollback.
Validasi	Bisa dilakukan manual via query.	Orphan FK, null violation, atau unique collision bisa luput.	Tambahkan validate_seed_integrity.

Prinsip desain

- 1 Plan first, write later. Semua seeding harus punya plan dan preview sebelum eksekusi.
- 2 Transaction by default. Jika satu table gagal, rollback seluruh plan.
- 3 Respect existing data. Pakai data parent yang sudah ada jika diminta, atau buat parent baru jika relasi membutuhkan.
- 4 Deterministic enough. Sediakan seed/random_seed agar output bisa diulang saat test.
- 5 Safe for development. Dry-run aktif secara default dan proteksi nama database production.

2. Tool inti yang paling disarankan

Prioritas	Tool	Fungsi utama	Kenapa penting
P0	plan_seed_data	Menganalisis target table, FK, constraint, dan membuat rencana dependency order.	Mengurangi hallucination agent. Agent tidak lagi menebak urutan insert.
P0	generate_seed_preview	Membuat contoh payload dummy berdasarkan plan tanpa menulis ke DB.	User bisa review sebelum data masuk.
P0	execute_seed_plan	Menjalankan seeding berdasarkan plan dengan batch insert dan transaction.	Membuat eksekusi lebih aman, cepat, dan atomik.
P0	validate_seed_integrity	Memvalidasi FK orphan, required column, unique collision, dan row count.	Membuktikan hasil seed benar, bukan hanya berhasil insert.
P1	infer_seed_rules	Menyimpulkan generator value dari schema, sample data, enum/check, dan nama kolom.	Meningkatkan kualitas data dummy untuk status, role, code, price, tanggal, email, dll.
P2	seed_from_template	Template seed reusable untuk domain seperti ecommerce, POS, CRM, LMS.	Mempercepat use case berulang, tetapi bukan wajib untuk MVP.

MVP paling ideal

Implementasikan P0 terlebih dahulu. Dengan 4 tool ini saja, MCP akan terasa punya dukungan native untuk relational seeding. P1 dan P2 bisa menyusul setelah pola data dan kebutuhan user lebih jelas.

Category permission baru

```
seed_operations = plan_seed_data, generate_seed_preview, execute_seed_plan, validate_seed_integrity, infer_seed_rules, seed_from_template
```

```
Minimal broad permissions: list,read,create,transaction
```

```
Jika execution butuh raw SQL internal: list,read,create,execute,transaction
```

Catatan: lebih aman jika execute_seed_plan memakai prepared insert/bulk insert internal, sehingga user tidak perlu membuka permission execute kepada agent untuk skenario seeding biasa.

3. Spesifikasi ringkas: plan_seed_data

Tool ini adalah pusat dari fitur seeding. Output-nya harus cukup kaya agar agent dapat menjelaskan rencana, meminta konfirmasi, lalu mengeksekusi tanpa perlu menebak ulang relasi.

Tujuan

- Menentukan semua tabel yang perlu disentuh untuk target seeding.
- Mengurutkan tabel berdasarkan dependency parent -> child.
- Mendeteksi FK, nullable, default, unique, enum/check jika tersedia, dan auto increment.
- Menentukan apakah harus memakai existing parent records atau membuat parent baru.
- Menghasilkan warning jika ada kolom/domain yang perlu keputusan user.

Contoh input

```
{
  "database": "dev_shop",
  "target_tables": ["orders"],
  "rows_per_table": 20,
  "include_dependencies": true,
  "respect_existing_data": true,
  "strategy": "append",
  "random_seed": 42
}
```

Contoh output

```
{
  "plan_id": "seed_plan_20260511_001",
  "dependency_order": ["users", "products", "orders", "order_items"],
  "tables_required": [
    {"table": "users", "rows_to_create": 10, "reason": "orders.user_id references users.id"},
    {"table": "products", "rows_to_create": 20, "reason": "order_items.product_id references products.id"},
    {"table": "orders", "rows_to_create": 20, "reason": "target table"},
    {"table": "order_items", "rows_to_create": 60, "reason": "child rows for orders"}
  ],
  "constraints_detected": {
    "foreign_keys": ["orders.user_id", "order_items.order_id", "order_items.product_id"],
    "unique": ["users.email", "products.sku"],
    "required_columns": ["orders.status", "orders.total"]
  },
  "warnings": [
    "orders.status appears domain-specific. Suggested values: pending, paid, cancelled."
  ],
  "requires_confirmation": true
}
```

4. Spesifikasi ringkas: generate_seed_preview

Tool ini membuat contoh data yang akan diinsert, tetapi tidak menulis ke database. Preview sangat penting karena data dummy sering salah bukan di sisi SQL, melainkan di sisi makna domain.

Contoh input

```
{
  "plan_id": "seed_plan_20260511_001",
  "locale": "id_ID",
  "realistic": true,
  "max_preview_rows_per_table": 3,
  "email_domain": "example.test"
}
```

Contoh output

```
{
  "plan_id": "seed_plan_20260511_001",
  "preview": {
    "users": [
      { "name": "Budi Santoso", "email": "budi.santoso.001@example.test" }
    ],
    "products": [
      { "name": "Keyboard Mechanical", "sku": "PRD-0001", "price": 350000 }
    ],
    "orders": [
      { "user_id": "{{users[0].id}}", "status": "pending", "total": 350000 }
    ],
    "order_items": [
      { "order_id": "{{orders[0].id}}", "product_id": "{{products[0].id}}", "qty": 1, "price": 350000 }
    ],
    "quality_notes": [
      "Generated email uses reserved example.test domain.",
      "Foreign keys are symbolic until execution resolves inserted IDs."
    ],
    "requires_confirmation": true
  }
}
```

Detail penting

- Gunakan placeholder simbolik seperti {{users[0].id}} agar relasi terlihat jelas sebelum insert.
- Jangan memakai domain email sungguhan; gunakan example.test atau domain yang disediakan user.
- Sediakan locale agar nama, alamat, nomor telepon, dan format tanggal bisa natural.
- Tambahkan deterministic random_seed dari plan agar preview dan execute konsisten.

5. Spesifikasi ringkas: `execute_seed_plan`

Tool ini menjalankan plan yang sudah dibuat. Default-nya harus aman: `dry_run true`, `transaction true`, `rollback on error`, dan `max row limit`.

Contoh input

```
{
  "plan_id": "seed_plan_20260511_001",
  "dry_run": false,
  "use_transaction": true,
  "batch_size": 500,
  "on_error": "rollback",
  "confirm_token": "CONFIRM_SEED_DEV_SHOP"
}
```

Contoh output

```
{
  "success": true,
  "transaction": "committed",
  "inserted": {
    "users": 10,
    "products": 20,
    "orders": 20,
    "order_items": 60
  },
  "resolved_foreign_keys": 80,
  "duration_ms": 842,
  "next_recommended_tool": "validate_seed_integrity"
}
```

Execution behavior yang disarankan

- 1 Cek guard environment: tolak database production kecuali `allow_production true` dan confirm token cocok.
- 2 Mulai transaction.
- 3 Insert tabel parent dulu sesuai `dependency_order`.
- 4 Tangkap inserted IDs dan resolve placeholder FK untuk child table.
- 5 Gunakan batch insert untuk row banyak.
- 6 Jika gagal, rollback dan kembalikan detail error per table/column.
- 7 Jika sukses, commit dan sarankan `validate_seed_integrity`.

Guard default yang sangat disarankan

```
{
  "dry_run_default": true,
  "require_confirmation": true,
  "allow_production_default": false,
  "blocked_database_name_patterns": ["prod", "production", "live", "main", "primary"],
  "max_rows_per_table_default": 1000,
  "use_transaction_default": true,
  "on_error_default": "rollback"
}
```

6. Spesifikasi ringkas: validate_seed_integrity

Tool ini melakukan pengecekan setelah insert. Tanpa validasi, agent hanya tahu query berhasil, bukan bahwa data dummy benar secara relasional.

Contoh input

```
{
  "plan_id": "seed_plan_20260511_001",
  "tables": ["users", "products", "orders", "order_items"],
  "check_foreign_keys": true,
  "check_orphans": true,
  "check_required_columns": true,
  "check_unique_collisions": true,
  "check_row_counts": true
}
```

Contoh output

```
{
  "valid": true,
  "summary": {
    "tables_checked": 4,
    "foreign_key_checks": 3,
    "orphan_foreign_keys": 0,
    "unique_violations": 0,
    "required_column_violations": 0
  },
  "row_counts": {
    "users": {"expected_inserted": 10, "actual_inserted": 10},
    "orders": {"expected_inserted": 20, "actual_inserted": 20}
  }
}
```

Validasi minimum

- FK orphan: child rows yang menunjuk parent yang tidak ada.
- Required columns: kolom NOT NULL tanpa default tidak boleh null.
- Unique collision: email, sku, code, slug, username, atau unique index lain tidak boleh bentrok.
- Row count: jumlah record inserted sesuai plan.
- Domain sanity: contoh total order sama dengan sum order_items jika rule tersedia.

7. Opsional: infer_seed_rules dan seed_from_template

Dua tool ini bukan MVP, tetapi akan membuat kualitas dummy data jauh lebih baik untuk project nyata.

infer_seed_rules

Tool ini membaca schema, constraint, sample rows, enum/check, serta pola nama kolom. Output-nya berupa rule generator per column. Ini membantu agent menghindari value ngawur seperti status yang tidak dikenal aplikasi.

```
{
  "rules": {
    "users.email": {"generator": "email", "unique": true, "domain": "example.test"},
    "users.name": {"generator": "person_name", "locale": "id_ID"},
    "orders.status": {"generator": "choice", "values": ["pending", "paid", "cancelled"]},
    "products.price": {"generator": "number", "min": 10000, "max": 500000},
    "products.sku": {"generator": "pattern", "pattern": "PRD-####", "unique": true}
  }
}
```

seed_from_template

Tool ini berguna untuk domain umum: ecommerce, POS, CRM, LMS, booking, helpdesk. Template dapat memanggil plan_seed_data di belakang layar lalu menerapkan pola table umum.

```
{
  "template": "ecommerce_basic",
  "scale": "small",
  "locale": "id_ID",
  "include": ["users", "products", "orders", "order_items", "payments"]
}
```

Namun untuk tahap awal, jangan mulai dari template. Mulai dari 4 tool inti agar desain stabil dan mudah diuji.

8. Contoh flow agent setelah tool ditambahkan

Dengan tool baru, prompt user pendek bisa diubah menjadi workflow aman dan eksplisit oleh agent.

```
User:
Buat dummy data 20 order lengkap dengan relasinya di database dev_shop.

Agent flow:
1. list_all_tools
2. plan_seed_data(target_tables=["orders"], rows_per_table=20, include_dependencies=true)
3. generate_seed_preview(plan_id, max_preview_rows_per_table=3)
4. Tampilkan preview dan minta konfirmasi
5. execute_seed_plan(plan_id, dry_run=false, use_transaction=true, on_error="rollback")
6. validate_seed_integrity(plan_id)
7. Laporkan inserted rows, warnings, dan validation result
```

Perbandingan sebelum vs sesudah

Aspek	Tanpa seed tools	Dengan seed_operations
Prompt pendek	Agent harus menebak banyak langkah.	Agent punya workflow standar.
FK order	Bisa salah jika relasi kompleks.	Dihitung oleh plan_seed_data.
Safety	Bergantung pada instruksi prompt.	Dry-run, confirmation, transaction, rollback by default.
Kualitas data	Bergantung pada LLM.	Didukung preview dan seed rules.
Validasi	Manual dan sering terlewat.	validate_seed_integrity menjadi langkah standar.

9. Rekomendasi implementasi teknis

Agar mudah di-maintain, implementasikan fitur ini sebagai orchestration layer di atas tool yang sudah ada, bukan mengganti bulk_insert atau transaction tools.

Architecture suggestion

- SeedPlanner: membangun graph FK, topological sort, dan table requirements.
- SeedRuleInferer: membaca schema, constraint, sample data, enum/check, dan column name patterns.
- SeedGenerator: menghasilkan row payload deterministic berdasarkan seed rules.
- SeedExecutor: menjalankan insert parent-child dalam transaction.
- SeedValidator: menjalankan post-check FK, unique, required column, dan row count.

Pseudocode sederhana

```
plan_seed_data(input):
    schema = get_database_summary(include_relationships=true)
    fk_graph = build_fk_graph(schema.relationships)
    required_tables = expand_dependencies(input.target_tables, fk_graph)
    order = topological_sort(required_tables, fk_graph)
    rules = infer_basic_rules(schema, required_tables)
    warnings = detect_ambiguous_required_columns(schema, rules)
    return { plan_id, dependency_order: order, rules, warnings }

execute_seed_plan(plan):
    assert_safe_environment(plan.database)
    begin_transaction()
    try:
        id_map = {}
        for table in plan.dependency_order:
            rows = generate_rows(table, plan.rules, id_map)
            inserted = bulk_insert(table, rows)
            id_map[table] = inserted.ids
        commit_transaction()
    except Error as err:
        rollback_transaction()
    return { success: false, error: err }
```

Testing checklist

- Single table tanpa FK.
- Parent-child sederhana: users -> orders.
- Many-to-many: orders -> order_items -> products.
- Composite unique index.
- Nullable vs NOT NULL columns.
- Existing parent data reused.
- Rollback saat child insert gagal.
- Database name mengandung prod/live harus ditolak secara default.

10. Kesimpulan final

MCP ini sudah baik sebagai MySQL MCP yang lengkap. Namun untuk case relational dummy data seeding, rekomendasi saya adalah menambahkan category seed_operations dengan 4 tool P0. Ini akan mengubah fitur dari “agent bisa melakukan kalau prompt bagus” menjadi “MCP memang support seeding relasional secara native”.

Keputusan	Rekomendasi
Apakah MCP sekarang sudah ok?	Ya, untuk access layer dan operasi DB umum.
Apakah cukup untuk seeding relasional otomatis?	Cukup untuk developer teknis, belum optimal untuk UX yang aman dan repeatable.
Tool wajib ditambah	plan_seed_data, generate_seed_preview, execute_seed_plan, validate_seed_integrity.
Tool opsional	infer_seed_rules dan seed_from_template.
Permission/category baru	seed_operations dengan default dry-run, confirmation, transaction, rollback, dan production guard.

Referensi

1. GitHub repository: <https://github.com/berthojoris/mysql-mcp> - dokumentasi README, permission system, kategori tool, dan daftar kemampuan MCP.
2. NPMX package index: <https://npmx.dev/package/%40berthojoris/mcp-mysql-server> - metadata package, versi 1.40.7, requirement Node.js, instalasi, dan ringkasan README.
3. GitHub raw documentation: <https://raw.githubusercontent.com/berthojoris/mysql-mcp/master/DOCUMENTATIONS.md> - dokumentasi tool seperti bulk operations, transaction management, database discovery, dan analysis.